

Prepared by Brian Kimtai

Loan Approval Prediction Using Decision Tree

Machine Learning Classification Project

This project uses a Decision Tree Classifier to predict whether a loan application should be approved or rejected based on applicant information such as:

- Income
- Education
- Employment status
- Loan amount
- Credit history
- Property area

The project demonstrates a complete machine learning workflow including:

- Data preprocessing
- Missing value handling
- Exploratory Data Analysis (EDA)
- Feature engineering
- Model training
- Model evaluation
- Feature importance analysis
- Decision Tree visualization

Problem Statement

Financial institutions receive a large number of loan applications every day.

Manual review of these applications is time-consuming, labor-intensive, and prone to inconsistency and human bias.

The objective of this project is to develop a Machine Learning model that can automatically predict whether a loan application should be approved or rejected based on applicant information.

This is a Binary Classification problem where:

- Y = Loan Approved
- N = Loan Rejected

Objectives

The objectives of this project are:

- Understand and explore the dataset
- Handle missing values
- Perform feature engineering
- Encode categorical variables
- Build a Decision Tree classifier
- Evaluate model performance
- Analyze feature importance
- Visualize the Decision Tree model

Import Libraries

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

from sklearn import tree

import warnings
warnings.filterwarnings('ignore')
```

Load Dataset

The dataset is loaded into a pandas DataFrame for analysis and preprocessing.

```
In [2]: # Load dataset
df = pd.read_csv("C:/Users/KIM/Downloads/train_loan.csv")

# Display first 5 rows
df.head()
```

Out[2]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
0	LP001002	Male	No	0	Graduate	No	5849	0	460	36	1	medium	A
1	LP001003	Male	Yes	1	Graduate	No	4583	0	460	36	1	medium	A
2	LP001005	Male	Yes	0	Graduate	Yes	3000	0	460	36	1	medium	A
3	LP001006	Male	Yes	0	Not Graduate	No	2583	0	460	36	1	medium	A
4	LP001008	Male	No	0	Graduate	No	6000	0	460	36	1	medium	A

Dataset Overview

The dataset contains applicant demographic and financial information together with the target variable Loan_Status.

The target variable indicates whether the loan was approved or rejected.

In [3]:

```
# Dataset shape
print("Dataset Shape:", df.shape)

# Dataset information
df.info()
```

Dataset Shape: (614, 13)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
Column Non-Null Count Dtype
--- -
0 Loan_ID 614 non-null object
1 Gender 601 non-null object
2 Married 611 non-null object
3 Dependents 599 non-null object
4 Education 614 non-null object
5 Self_Employed 582 non-null object
6 ApplicantIncome 614 non-null int64
7 CoapplicantIncome 614 non-null float64
8 LoanAmount 592 non-null float64
9 Loan_Amount_Term 600 non-null float64
10 Credit_History 564 non-null float64
11 Property_Area 614 non-null object
12 Loan_Status 614 non-null object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB

Dataset Information Interpretation

The dataset contains 614 loan application records with 13 variables.

The variables include both numerical and categorical features related to applicant financial status, demographic information, and loan details.

Several columns contain missing values, including:

- Gender
- Married
- Dependents
- Self_Employed
- LoanAmount
- Loan_Amount_Term
- Credit_History

These missing values must be handled before building the machine learning model.

Statistical Summary

The statistical summary helps understand the distribution of numerical variables in the dataset.

In [4]:

```
# Statistical summary
df.describe()
```

Out[4]:

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_Hist
count	614.000000	614.000000	592.000000	600.00000	564.0000
mean	5403.459283	1621.245798	146.412162	342.00000	0.8421
std	6109.041673	2926.248369	85.587325	65.12041	0.3648
min	150.000000	0.000000	9.000000	12.00000	0.0000
25%	2877.500000	0.000000	100.000000	360.00000	1.0000
50%	3812.500000	1188.500000	128.000000	360.00000	1.0000
75%	5795.000000	2297.250000	168.000000	360.00000	1.0000
max	81000.000000	41667.000000	700.000000	480.00000	1.0000

Statistical Summary Interpretation

Key observations include:

- The average applicant income is approximately 5403.
- The average loan amount is approximately 146.

- Most loan terms are around 360 months.
- Many applicants have a positive credit history.

The dataset also contains possible outliers in income values.

Missing Values Analysis

This section identifies missing values present in the dataset.

```
In [5]: # Check missing values
df.isnull().sum()
```

```
Out[5]: Loan_ID          0
Gender          13
Married         3
Dependents      15
Education        0
Self_Employed   32
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount      22
Loan_Amount_Term 14
Credit_History  50
Property_Area    0
Loan_Status      0
dtype: int64
```

Missing Values Interpretation

Several columns contain missing values.

The highest number of missing values appears in:

- Credit_History
- Self_Employed
- LoanAmount

Missing values can negatively affect model performance if not handled properly.

Handling Missing Values

To prepare the dataset for machine learning:

- Categorical variables are filled using the mode
- Numerical variables are filled using the mean

Handle Categorical Missing Values

```
In [6]: # Fill categorical missing values using mode

df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])

df['Married'] = df['Married'].fillna(df['Married'].mode()[0])

df['Dependents'] = df['Dependents'].fillna(df['Dependents'].mode()[0])

df['Self_Employed'] = df['Self_Employed'].fillna(df['Self_Employed'].mode()[0])
```

Handle Numerical Missing Values

```
In [7]: # Fill numerical missing values using mean

df['LoanAmount'] = df['LoanAmount'].fillna(df['LoanAmount'].mean())

df['Loan_Amount_Term'] = df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mean())

df['Credit_History'] = df['Credit_History'].fillna(df['Credit_History'].mean())
```

Confirm Missing Values Are Removed

```
In [8]: df.isnull().sum()
```

```
Out[8]: Loan_ID          0
Gender          0
Married         0
Dependents      0
Education       0
Self_Employed   0
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount      0
Loan_Amount_Term 0
Credit_History  0
Property_Area   0
Loan_Status     0
dtype: int64
```

Missing Values Handling Interpretation

After preprocessing, all missing values were successfully removed from the dataset.

The dataset is now clean and ready for analysis and machine learning modeling.

Exploratory Data Analysis (EDA)

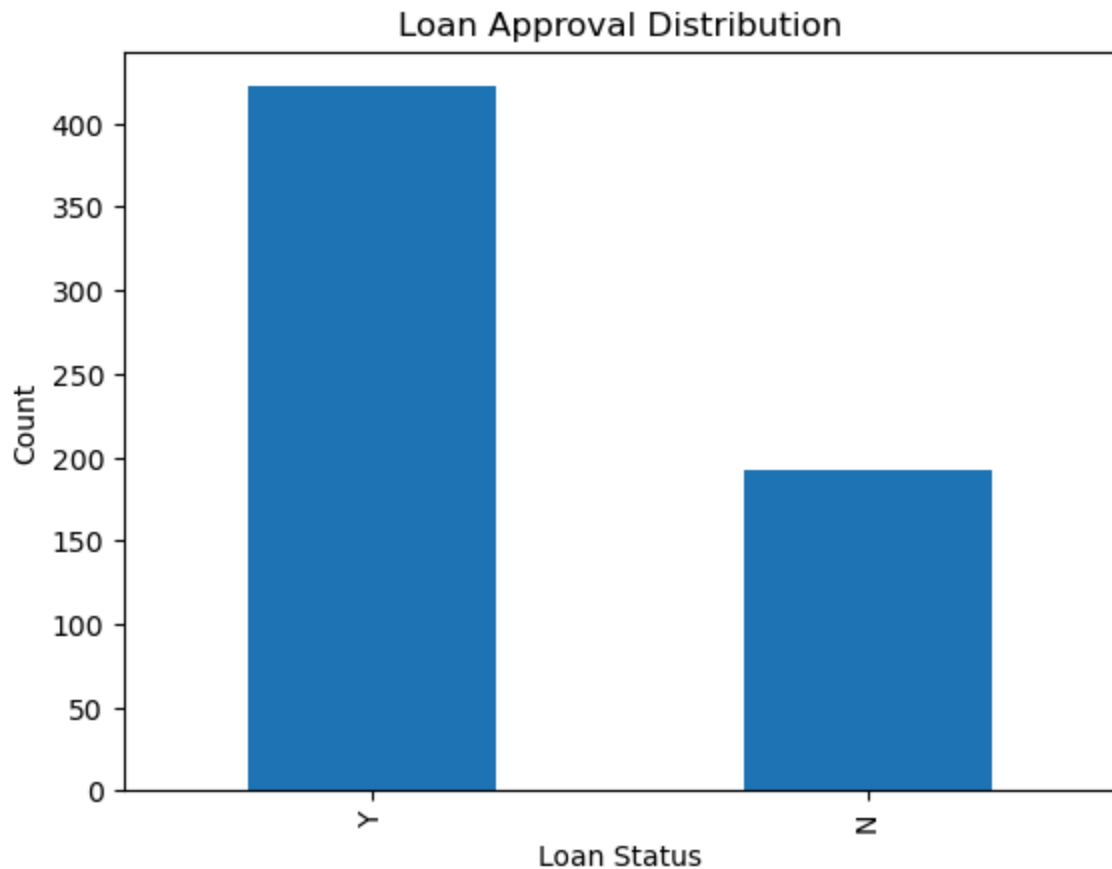
EDA helps understand relationships, trends, and distributions within the dataset.

Loan Approval Distribution

```
In [9]: df['Loan_Status'].value_counts().plot(kind='bar')

plt.title("Loan Approval Distribution")
plt.xlabel("Loan Status")
plt.ylabel("Count")

plt.show()
```



Loan Approval Distribution Interpretation

The number of approved loans is higher than the number of rejected loans.

This indicates slight class imbalance in the dataset.

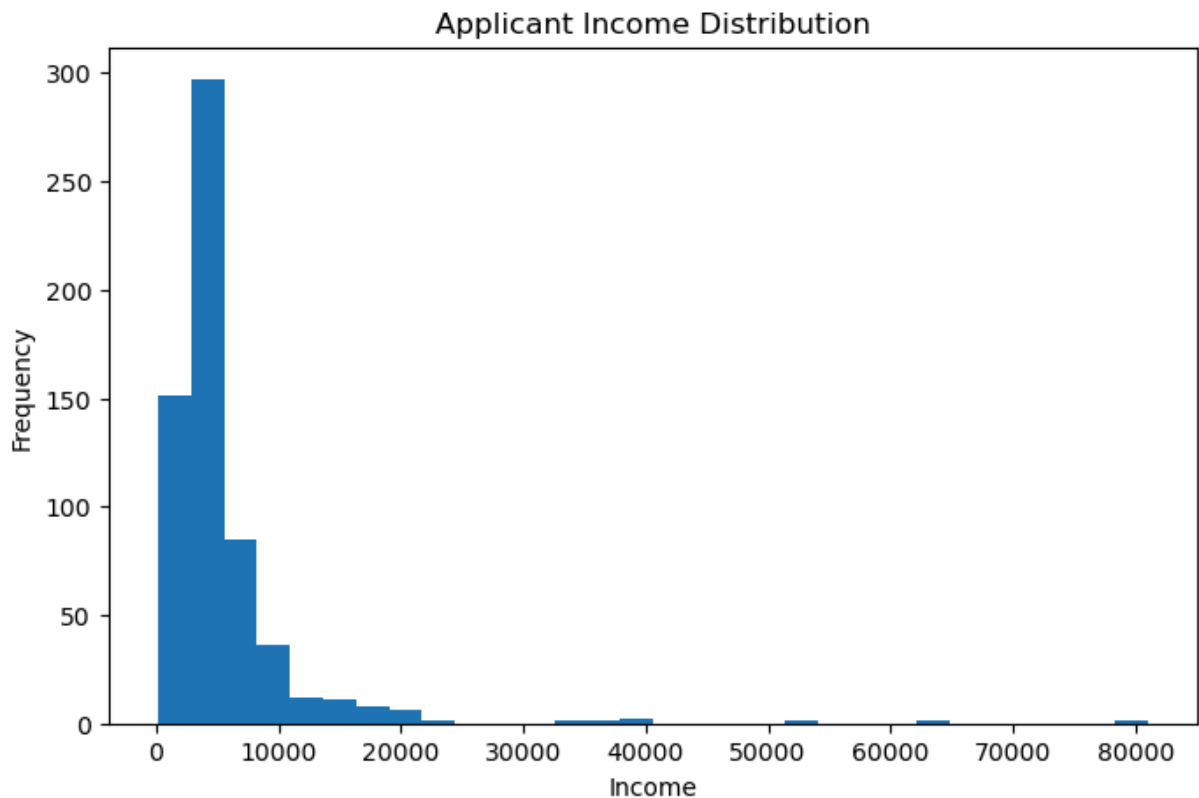
Applicant Income Distribution

```
In [10]: plt.figure(figsize=(8,5))

plt.hist(df['ApplicantIncome'], bins=30)

plt.title("Applicant Income Distribution")
plt.xlabel("Income")
plt.ylabel("Frequency")

plt.show()
```



Applicant Income Interpretation

The income distribution is positively skewed.

Most applicants have moderate income levels while a few applicants have very high incomes.

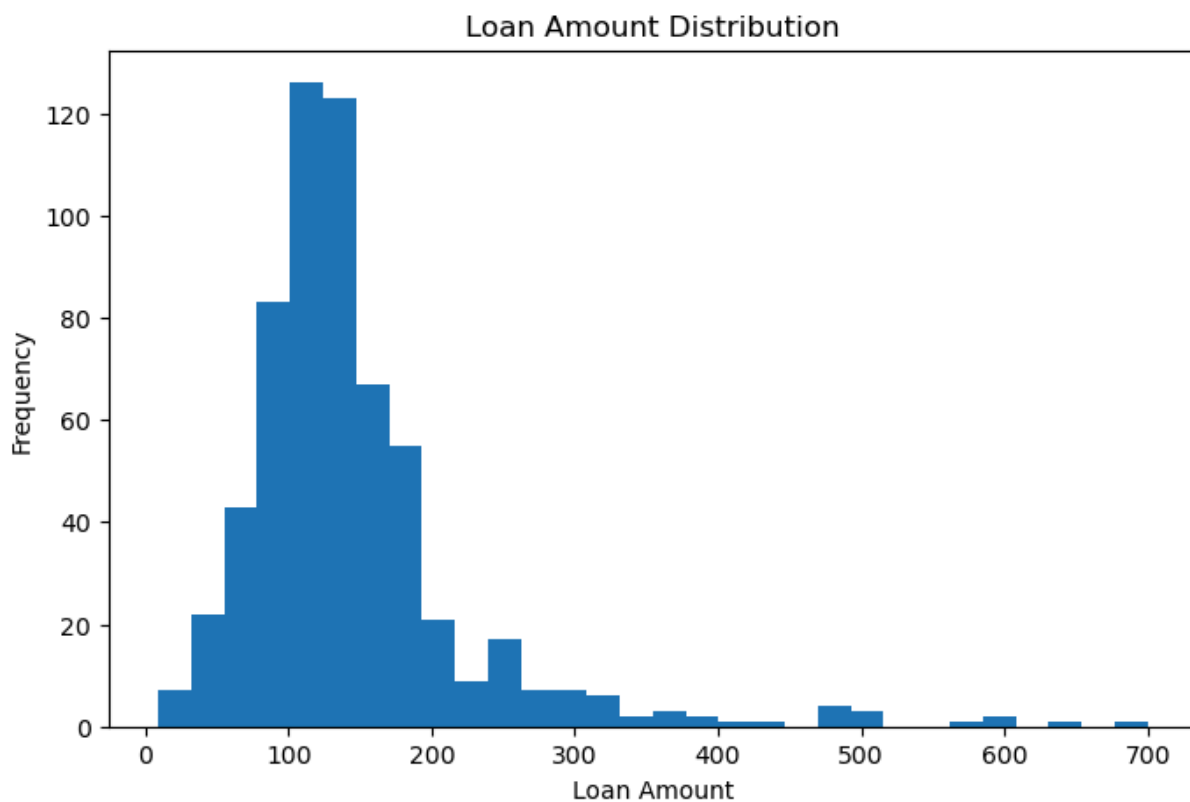
Loan Amount Distribution

```
In [11]: plt.figure(figsize=(8,5))

plt.hist(df['LoanAmount'], bins=30)

plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")

plt.show()
```



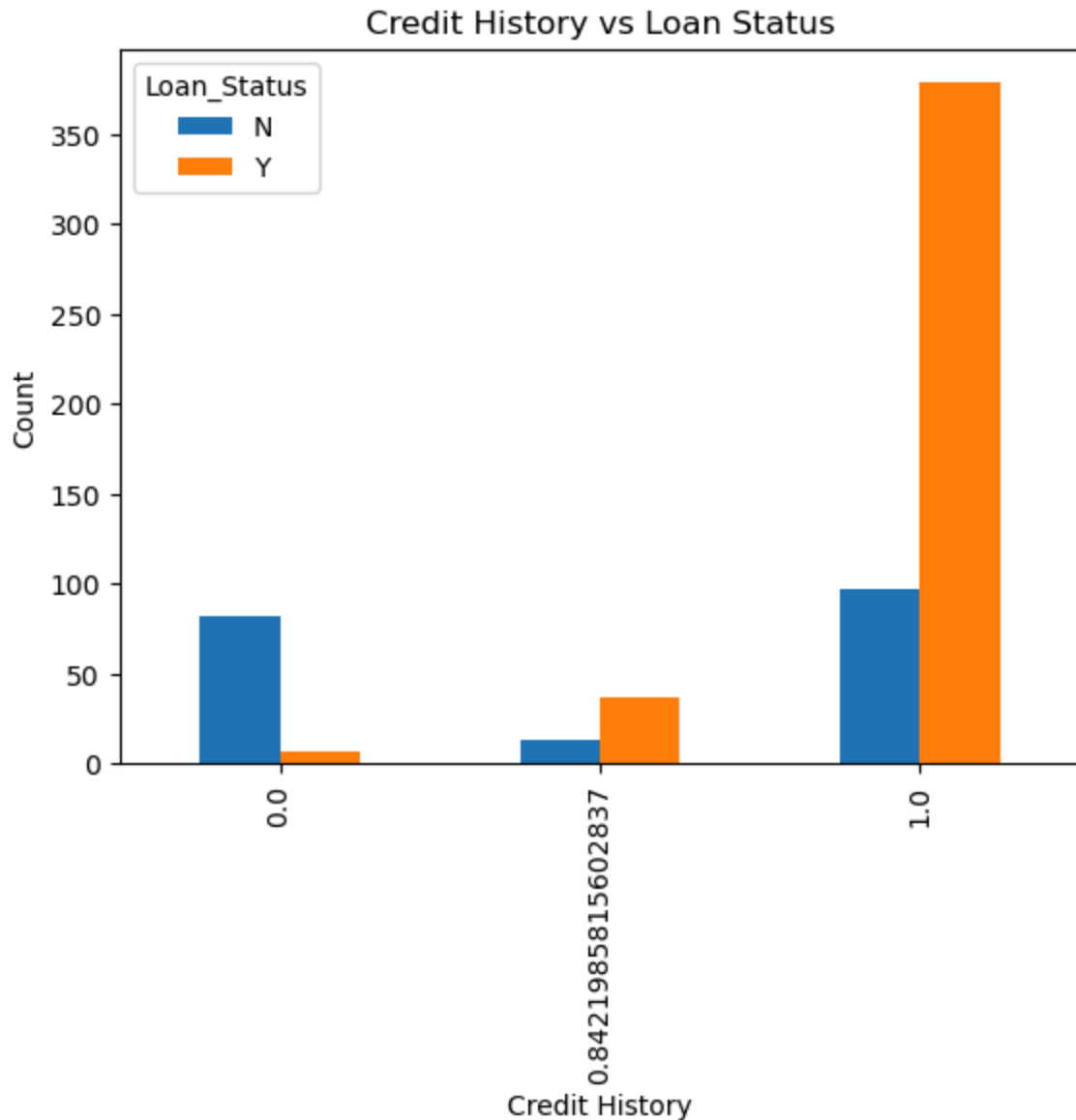
Loan Amount Distribution

Credit History vs Loan Status

```
In [12]: pd.crosstab(df['Credit_History'], df['Loan_Status']).plot(kind='bar')

plt.title("Credit History vs Loan Status")
plt.xlabel("Credit History")
plt.ylabel("Count")

plt.show()
```



Credit History Interpretation

Applicants with positive credit history are more likely to receive loan approval.

This suggests that credit history is a major factor influencing loan approval decisions.

Feature Engineering

Feature engineering creates new meaningful variables that may improve model performance.

```
In [13]: # Create Total Income feature
df['Total_Income'] = (
    df['ApplicantIncome'] + df['CoapplicantIncome']
)
```

```
# Loan to Income Ratio
df['Loan_to_Income_Ratio'] = (
    df['LoanAmount'] / df['Total_Income']
)

# Estimated Monthly Installment
df['EMI'] = (
    df['LoanAmount'] / df['Loan_Amount_Term']
)

print("Feature Engineering Completed")
```

Feature Engineering Completed

Feature Engineering Interpretation

The engineered features provide additional financial insights about applicants and may improve prediction performance.

Encoding Categorical Variables

Machine learning models require numerical input data.

Categorical variables are converted using One-Hot Encoding.

```
In [14]: categorical_columns = [
    'Gender',
    'Married',
    'Dependents',
    'Education',
    'Self_Employed',
    'Property_Area'
]

# One-Hot Encoding
df = pd.get_dummies(
    df,
    columns=categorical_columns,
    drop_first=True
)
```

Encoding Interpretation

Categorical variables were successfully converted into numerical format.

Target Variable Encoding

The target variable is converted into numerical format.

```
In [15]: # Clean target variable
df['Loan_Status'] = df['Loan_Status'].str.strip()

# Convert target variable to numeric
df['Loan_Status'] = df['Loan_Status'].map({
    'Y': 1,
    'N': 0
})
```

Target Variable Interpretation

- Approved loans = 1
- Rejected loans = 0

This enables the Decision Tree model to perform classification.

Define Features and Target Variable

```
In [16]: # Define X and y

X = df.drop(columns=['Loan_Status', 'Loan_ID'])

y = df['Loan_Status']

print("X Shape:", X.shape)
print("y Shape:", y.shape)
```

X Shape: (614, 17)

y Shape: (614,)

Train-Test Split

The dataset is divided into training and testing subsets.

- Training set = 80%
- Testing set = 20%

```
In [17]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

Training Shape: (491, 17)

Testing Shape: (123, 17)

Train-Test Split Interpretation

The training dataset is used for model learning while the testing dataset evaluates model performance on unseen data.

Build Decision Tree Model

A Decision Tree Classifier is used to predict loan approval status.

```
In [18]: # Create Decision Tree model

model = DecisionTreeClassifier(
    max_depth=6,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

print("Model Trained Successfully")
```

Model Trained Successfully

Model Training Interpretation

The Decision Tree model successfully learned patterns between applicant characteristics and loan approval outcomes.

Make Predictions

```
In [19]: # Make predictions
predictions = model.predict(X_test)
```

Model Accuracy

```
In [20]: # Calculate accuracy

accuracy = accuracy_score(y_test, predictions)
```

```
print("Accuracy:", round(accuracy * 100, 2), "%")
```

Accuracy: 84.55 %

Accuracy Interpretation

The Decision Tree model achieved an accuracy of approximately 84.55%.

This means that the model correctly predicted the loan approval status for about 85 out of every 100 applicants in the testing dataset.

The result indicates that the model has strong predictive performance and is able to identify patterns associated with loan approval decisions effectively.

The high accuracy suggests that variables such as:

- Credit History
- Applicant Income
- Loan Amount
- Loan-to-Income Ratio

play an important role in determining loan approval outcomes.

Overall, the model performs well and demonstrates that Decision Tree algorithms can be useful in supporting automated loan approval systems in financial institutions.

Classification Report

```
In [21]: print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
0	0.83	0.63	0.72	38
1	0.85	0.94	0.89	85
accuracy			0.85	123
macro avg	0.84	0.79	0.81	123
weighted avg	0.84	0.85	0.84	123

Classification Report Interpretation

The model achieved strong precision and recall scores.

The results indicate that the model performs well in distinguishing approved and rejected loan applications.

Confusion Matrix

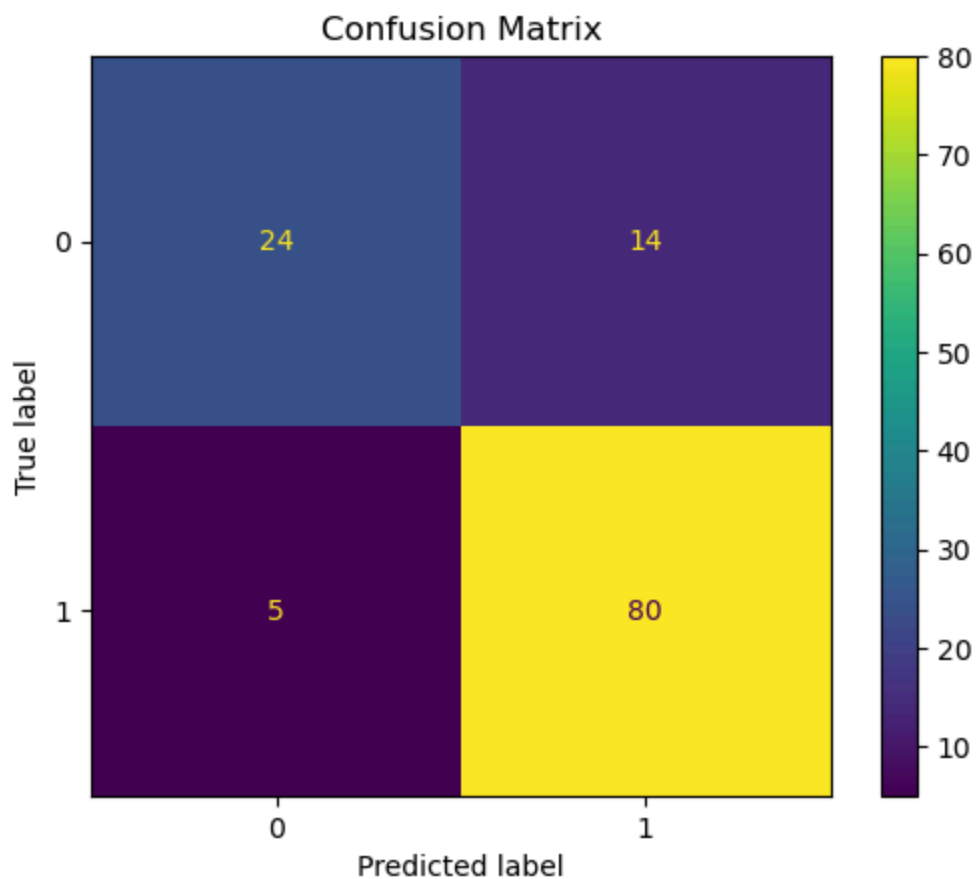
```
In [22]: cm = confusion_matrix(y_test, predictions)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm
)

disp.plot()

plt.title("Confusion Matrix")

plt.show()
```



Confusion Matrix Interpretation

The confusion matrix shows that most applications were classified correctly.

Only a small number of applications were misclassified.

Feature Importance Analysis

```
In [23]: # Feature importance

importance = pd.Series(
    model.feature_importances_,
    index=X.columns
)

importance = importance.sort_values(
    ascending=True
)

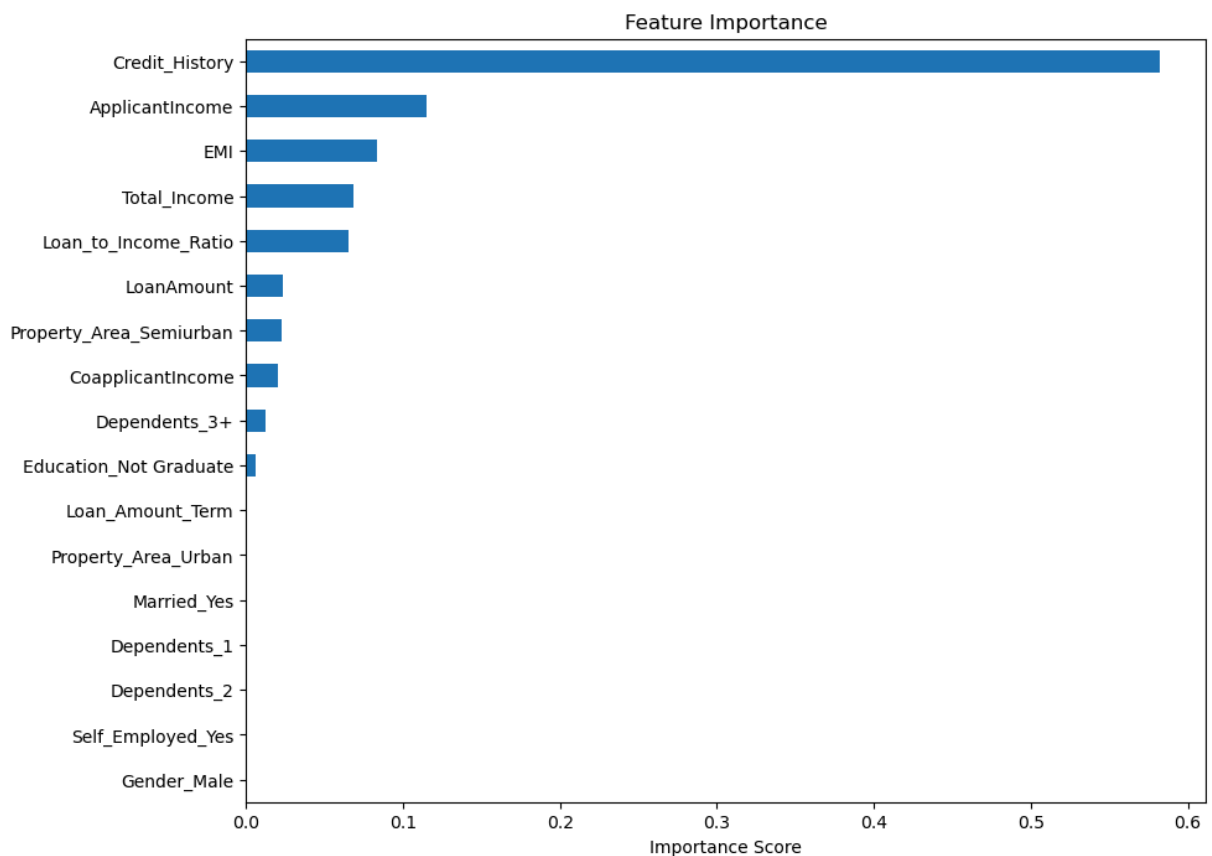
# Plot feature importance

plt.figure(figsize=(10,8))

importance.plot(kind='barh')

plt.title("Feature Importance")
plt.xlabel("Importance Score")

plt.show()
```



Feature Importance Interpretation

Credit_History is the most influential feature in predicting loan approval.

Other important variables include:

- Total Income
- Loan Amount
- Loan-to-Income Ratio

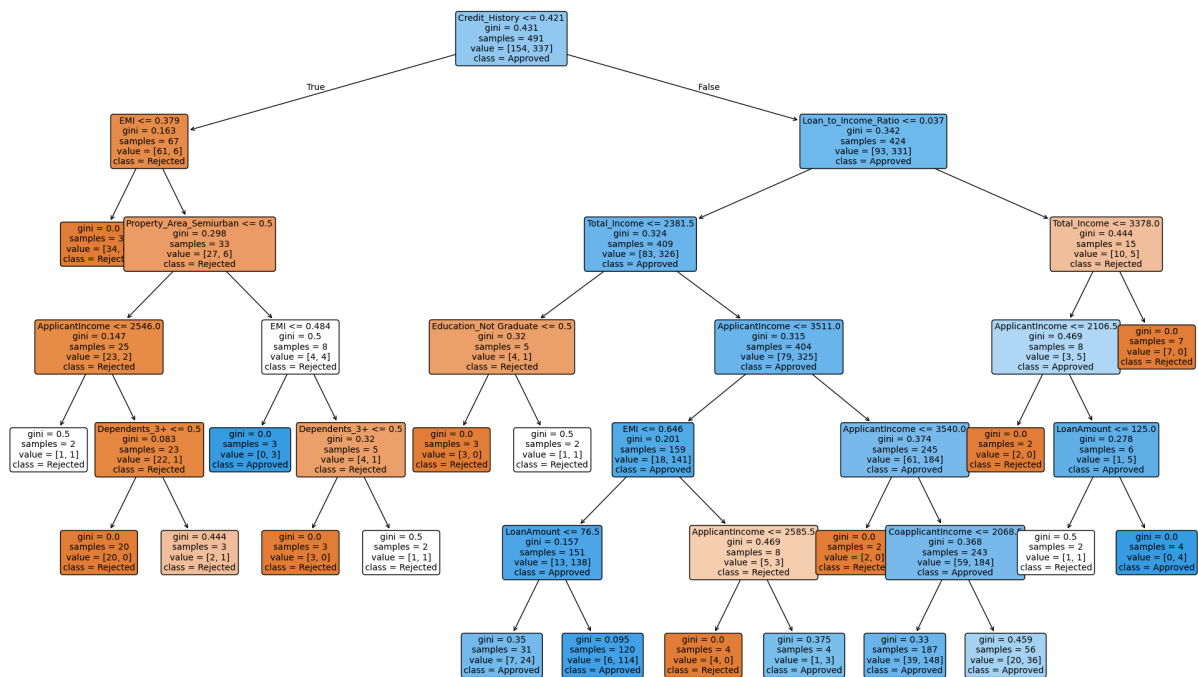
These findings align with real-world banking practices.

Decision Tree Visualization

```
In [24]: plt.figure(figsize=(25,15))

tree.plot_tree(
    model,
    filled=True,
    rounded=True,
    fontsize=10,
    feature_names=X.columns,
    class_names=['Rejected', 'Approved']
)

plt.show()
```



Decision Tree Visualization Interpretation

The visualization illustrates how the model makes decisions based on applicant characteristics.

Each branch represents a decision rule used during prediction.

Conclusion

This project successfully developed a Decision Tree model for predicting loan approval status.

The model achieved approximately 84.55% accuracy, demonstrating strong predictive performance.

Key findings include:

- Credit History was the most important feature
- Feature engineering improved prediction quality
- Decision Trees can effectively support financial decision-making

The project demonstrates the practical application of Machine Learning in automating loan approval systems.